DBMS & SQL REVISION GUIDE

A Mechanism-Driven Handbook on Database Normalization, B+ Trees, ACID, MVCC & Query Optimization

A Concise, Visual, and Mechanism-Focused Interview & Revision Handbook

METADATA FIELD	SPECIFICATION
Subject:	Database Management Systems & SQL
Target Audience:	SDE / Technical Interviews (Top Tech, FAANG, Product Firms)
Methodology:	Internal Mechanics • State Transitions • Verified Code Recipes • Trade-offs
Source File Consolidated:	Interview Concepts/Databases.sql (Normalization 1NF-BCNF, Indexing on columns)
Subject Coverage:	Relational Schemas, Normalization, B+ Tree Indexing, ACID, Concurrency Anomalies, MVCC, SQL & Sharding
Key Architectural Focus:	Isolation Anomalies Matrix, Leftmost Prefix Rule, Lock Hierarchies & Query Execution Order
Target Evaluation:	Technical Interview Placement Preparation - Core Engineering Track

CH	CHAPTER TITLE & CORE MECHANISM	CRITICAL INTERVIEW CONCEPTS & THEOREMS
1	Relational Models, Keys & Integrity Constraints	Primary, Candidate, Foreign, Unique, Check; Entity & Referential integrity
2	Database Normalization: 1NF to BCNF	Functional Dependencies, 1NF Atomic, 2NF Partial, 3NF Transitive, BCNF
3	Indexing Mechanics: B-Trees, B+ Trees & Composite	B+ Tree structure, Clustered vs Secondary, Leftmost Prefix Rule, Write penalty
4	Transactions & ACID Properties	Atomicity (WAL), Consistency, Isolation, Durability; Commit & Rollback
5	Concurrency Anomalies & Isolation Levels	Dirty Read, Non-Repeatable Read, Phantom Read; Read Committed to Serializable
6	Concurrency Control: 2PL Locking vs MVCC	Shared vs Exclusive Locks, Growing/Shrinking 2PL, MVCC Snapshots & Non-blocking Reads
7	SQL Query Mastery & Logical Execution Order	FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY; Joins & Windowing
8	Database Scaling: Replication, Sharding & CAP	Leader-Follower, Read Replicas, Horizontal Sharding, CAP Theorem trade-offs

CHAPTER 1 - RELATIONAL MODELS, KEYS & INTEGRITY CONSTRAINTS

A Relational Database represents data as mathematical relations (tables) consisting of tuples (rows) and attributes (columns). Keys enforce structural uniqueness and define associations across tables.

1.1 Database Keys Taxonomy

Key Type	Definition & Rule	Null Permitted?	Typical Application
Super Key	Any set of attributes that uniquely identifies a row in a table.	Depends on columns	Broadest uniqueness set (e.g., {SSN, Name, Phone}).
Candidate Key	A **minimal Super Key** with no redundant attributes.	NO (for primary candidate)	Candidate options from which the Primary Key is selected.
Primary Key	The chosen candidate key for unique row identification.	**STRICTLY NO**	Indexed cluster key for fast point lookups.
Alternate Key	Any candidate key NOT selected as the Primary Key.	Depends	Secondary candidate (e.g., unique email or phone number).
Foreign Key	An attribute in Table A that references the Primary Key of Table B.	YES (unless specified NOT NULL)	Enforces Referential Integrity across parent-child relationships.

1.2 Referential Integrity & Cascade Actions

Referential Integrity dictates that a Foreign Key value must always reference a valid, existing Primary Key in the parent table (or be NULL).

- ON DELETE CASCADE: Deleting parent row automatically deletes all referencing child rows.
- ON DELETE SET NULL: Sets foreign key to NULL upon parent deletion (requires FK column to be nullable).
- ON DELETE RESTRICT / NO ACTION: Rejects parent deletion if any dependent child rows exist.

CHAPTER 2 - DATABASE NORMALIZATION: 1NF TO BCNF

Normalization decomposes tables to eliminate update, insertion, and deletion anomalies while minimizing data redundancy.

2.1 The Normal Forms Progression

Normal Form	Core Requirement to Satisfy	Violating Anomaly Prevented
1st Normal Form (1NF)	All attributes must contain **Atomic (indivisible) values**; no repeating groups or arrays.	Inability to query or index array items directly (e.g., comma-separated phones).
2nd Normal Form (2NF)	Must be in 1NF AND **No Partial Dependency**: every non-prime attribute must depend on the *entire* candidate key, not a proper subset.	Update anomaly: changing a course name requires updating multiple student-course rows.
3rd Normal Form (3NF)	Must be in 2NF AND **No Transitive Dependency**: non-prime attributes must depend ONLY on the primary key (X -> Y and Y -> Z where Z is non-prime is banned).	Insertion anomaly: cannot record department name without hiring an employee.
Boyce-Codd (BCNF)	Strict 3NF: For every functional dependency X -> Y, **X must be a Super Key**.	Redundancy where overlapping candidate keys exist.

2.2 Normalization Decision Rule (The Kent Rule)

THE SACRED NORMALIZATION RULE

'Every non-key attribute must provide a fact about the key (1NF), the whole key (2NF), and nothing but the key (3NF), so help me Codd.'

- If an attribute depends on part of a composite key -> **Violates 2NF**. Split into separate parent table.
- If an attribute depends on another non-key attribute -> **Violates 3NF**. Split into lookup table.

CHAPTER 3 - INDEXING MECHANICS: B+ TREES & COMPOSITE INDEXES

An Index is an auxiliary search data structure that enables the database engine to find specific rows in O(log N) disk I/O operations rather than performing an expensive sequential full table scan (O(N)).

3.1 Why Relational Databases Use B+ Trees (Not Binary Search Trees)

1. **High Fan-Out & Low Tree Height:** BSTs have a fan-out of 2, requiring 20-30 disk seeks for 1M rows. A B+ Tree node matches the disk page size (typically 16 KB) with a fan-out of 100-500, keeping tree height to just 3-4 levels (only 3-4 disk seeks!).
2. **Sequential Linked Leaf Nodes:** In a B+ Tree, internal nodes store only keys (routing pointers), while ALL data records reside in leaf nodes. Leaves are doubly linked, making range scans (BETWEEN X AND Y) ultra-fast sequential scans without re-traversing the tree.

3.2 Clustered vs Secondary (Non-Clustered) Index

Dimension	Clustered Index	Secondary (Non-Clustered) Index
Physical Storage	Dictates the **physical sorting order of rows on disk**.	Separate structure containing index keys and row pointers.
Limit per Table	**Exactly ONE** per table (table can only be physically sorted one way!).	Multiple allowed per table.
Leaf Node Payload	Contains the **actual full table row data**.	Contains the Clustered Key value (in InnoDB) or row pointer (RID).
Lookup Performance	Instant point fetch (no secondary hop).	Requires two lookups: secondary index -> primary key -> row data (unless covered!).

3.3 Composite Indexes & The Leftmost Prefix Rule

Code: Composite Indexing & Leftmost Prefix Rule

```
-- Creating a Composite Index (from Databases.sql)
CREATE INDEX idx_name_age ON students(name, age);

-- Leftmost Prefix Rule: Index is physically sorted by name FIRST, then by age SECOND!
SELECT * FROM students WHERE name = 'Alice';               -- USES INDEX! (Leftmost prefix)
SELECT * FROM students WHERE name = 'Alice' AND age = 21;  -- USES INDEX FULLY!
SELECT * FROM students WHERE age = 21;                    -- CANNOT USE INDEX! (Missing leftmost 'name')
```

CHAPTER 4 - TRANSACTIONS & ACID PROPERTIES

A **Transaction** is a logical unit of database work consisting of one or more SQL statements that execute as a single indivisible operation.

Property	Meaning / Guarantee	Internal Database Implementation
A - Atomicity	All-or-Nothing: Either every modification succeeds and commits, or entire transaction aborts and rolls back.	**Write-Ahead Logging (WAL) / Undo Log:** Undo log stores inverse operations to reverse partial writes on crash.
C - Consistency	State transitions from one valid state to another, satisfying all schema constraints, foreign keys, and triggers.	Application logic combined with DB constraint validation (NOT NULL, CHECK, UNIQUE).
I - Isolation	Concurrent transactions execute without interfering with one another; uncommitted intermediate states remain invisible.	**Locking (2PL)** or **Multi-Version Concurrency Control (MVCC)** using snapshot reads.
D - Durability	Once committed, changes survive permanently even if power failure, crash, or reboot occurs immediately after.	**Redo Log / WAL:** Committed log records are flushed to disk (fsync) before returning success to client.

WRITE-AHEAD LOGGING (WAL) INVARIANT

A database engine NEVER writes modified data pages directly to table data files on disk during a transaction commit! Writing dirty pages randomly across disk is too slow. Instead, it appends a compact sequential log entry to the **Write-Ahead Log (WAL)**. *Invariant:* Log records must hit persistent disk BEFORE data pages are written. On recovery, the engine replays the redo log.

CHAPTER 5 - CONCURRENCY ANOMALIES & ISOLATION LEVELS

5.1 The 4 Classic Concurrency Anomalies

• **Dirty Read:** Transaction T1 reads data modified by T2 that has NOT YET committed. If T2 subsequently rolls back, T1 processed phantom invalid data.
• **Non-Repeatable Read (Fuzzy Read):** T1 reads row X. T2 updates/deletes row X and commits. T1 re-reads row X and discovers value has changed.
• **Phantom Read:** T1 queries a range of rows satisfying a condition (e.g., WHERE age > 20 -> returns 5 rows). T2 inserts a new row matching that condition and commits. T1 re-executes query and gets 6 rows.
• **Lost Update:** T1 and T2 read same balance ($100). Both modify and save ($150 and $120). One overwrite silently obliterates the other update.

5.2 SQL Standard Isolation Levels Matrix

Isolation Level	Dirty Read?	Non-Repeatable Read?	Phantom Read?	Locking / Mechanism
Read Uncommitted	**YES (Permitted)**	**YES**	**YES**	No read locks; reads latest dirty memory pages.
Read Committed	NO (Prevented)	**YES**	**YES**	Reads snapshot taken at start of each statement. Default in Postgres/Oracle.
Repeatable Read	NO	NO (Prevented)	**YES (in SQL spec)***	Reads snapshot taken at start of transaction. Default in MySQL InnoDB (uses Next-Key locks to prevent Phantoms!).
Serializable	NO	NO	NO (Prevented)	Strict serial execution order guaranteed via range locks or SSI.

CHAPTER 6 - CONCURRENCY CONTROL: 2PL LOCKING VS MVCC

6.1 Two-Phase Locking (2PL)

2PL guarantees serializability by dividing lock management into two monotonic phases:
1. **Growing Phase:** Transaction acquires locks; cannot release any lock.
2. **Shrinking Phase:** Once a lock is released, transaction enters shrinking phase and CANNOT acquire any further locks.
• **Strict 2PL:** All exclusive (write) locks are held until transaction commits/aborts (prevents cascading aborts).

6.2 Multi-Version Concurrency Control (MVCC)

Core Principle: *'Readers never block writers, and writers never block readers.'*
Instead of locking rows, each row update creates a new immutable version tagged with transaction IDs (xmin, xmax). A reading transaction reads the historical version visible to its snapshot timestamp without acquiring any read locks!

Dimension	Pessimistic Locking (2PL)	MVCC (Snapshot Isolation)
Read-Write Conflict	Reads block writes; writes block reads.	Reads do NOT block writes; writes do NOT block reads.
Concurrency Throughput	Low on read-heavy systems under high contention.	Extremely high concurrent read performance.
Storage Overhead	Minimal (locks tracked in RAM lock tables).	Requires vacuuming / purge threads to clean dead old row versions (PostgreSQL VACUUM).

CHAPTER 7 - SQL QUERY MASTERY & LOGICAL EXECUTION ORDER

7.1 The True Logical Execution Order of SQL

While SQL is written starting with SELECT, the SQL engine executes clauses in strict mathematical sequence:

1. **FROM & JOIN:** Identify base tables, cross-product, and apply join conditions.
2. **WHERE:** Filter rows before aggregation (cannot filter aggregate functions here!).
3. **GROUP BY:** Partition rows into aggregate buckets.
4. **HAVING:** Filter aggregate groups (operates on post-grouping metrics).
5. **SELECT:** Evaluate expressions, column aliases, and window functions.
6. **DISTINCT:** Deduplicate rows.
7. **ORDER BY:** Sort the final result set.
8. **LIMIT / OFFSET:** Restrict returned rows.

7.2 Window Functions: ROW_NUMBER vs RANK vs DENSE_RANK

Function	Tie Handling	Example for Values [100, 100, 80]	Use Case
ROW_NUMBER()	Assigns sequential unique integers; breaks ties arbitrarily.	1, 2, 3	Deduplicating rows, pagination.
RANK()	Equal values get same rank; **skips subsequent ranks**.	1, 1, 3 (skips 2!)	Leaderboards with rank skipping.
DENSE_RANK()	Equal values get same rank; **NO gaps in sequence**.	1, 1, 2	Nth highest salary query.

7.3 Essential Interview SQL Patterns

Code: Top SQL Interview Query Patterns

```
-- 1. Find 2nd Highest Salary (Dense Rank Pattern)
WITH RankedSalaries AS (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
)
SELECT salary FROM RankedSalaries WHERE rnk = 2 LIMIT 1;

-- 2. Department-Wise Top Earner (Partition By)
SELECT emp_name, dept_id, salary
FROM (
    SELECT emp_name, dept_id, salary,
           ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) as rk
    FROM employees
) t WHERE rk = 1;
```

CHAPTER 8 - DATABASE SCALING: REPLICATION, SHARDING & CAP

8.1 Master-Slave (Leader-Follower) Replication

• **Primary (Leader):** Handles all write operations (INSERT, UPDATE, DELETE) and streams changes via replication log.
• **Replicas (Followers):** Replay replication stream and serve read-only queries. Scales read throughput horizontally.
• **Replication Lag Traps:** In asynchronous replication, reading immediately after writing can return stale data (solved by *Read-Your-Own-Writes Consistency* via routing user's own reads to primary for 5 seconds).

8.2 Horizontal Sharding vs Partitioning

When data exceeds single-server disk/RAM capacity, **Sharding** distributes rows across independent database nodes using a **Shard Key**.
• **Hash-Based Sharding:** node = hash(user_id) % num_nodes. Even distribution, but reshuffling all data when adding nodes (mitigated via **Consistent Hashing**).
• **Range-Based Sharding:** Partitions by contiguous ranges (e.g., Dates, A-Z). Risk of hotspots (all writes hit current month shard!).

8.3 CAP Theorem in Distributed Databases

Property	Meaning	Network Partition Trade-off
Consistency (C)	Every read receives the most recent write or an error.	CP System: If network partitions, reject writes to maintain consistency (HBase, MongoDB).
Availability (A)	Every non-failing node returns a response, without guarantee it contains latest write.	AP System: If partition occurs, accept writes on both sides; reconcile later (Cassandra, DynamoDB).
Partition Tolerance (P)	System continues functioning despite network packet drops or delays.	**MANDATORY:** Network partitions are physical realities in distributed systems; you MUST choose CP or AP!

DBMS & SQL INTERVIEW CHEAT-SHEET
• **Normalization:** 1NF (Atomic), 2NF (Full functional dependency), 3NF (No transitive dependency), BCNF (Determinants are Candidate Keys).
• **B+ Trees:** High fan-out keeps depth to 3-4 levels; doubly-linked leaves enable O(1) range scans.
• **Leftmost Prefix:** Composite index on (A, B) cannot service queries filtering solely on B.
• **ACID Mechanisms:** Atomicity = Undo Log; Durability = Redo Log/WAL; Isolation = 2PL or MVCC.
• **Logical SQL Order:** FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY.